CODA

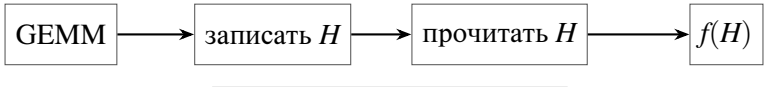
Блоки Transformer как программы GEMM + эпилог

Меньше обмена с памятью GPU
при прямом и обратном проходах.

Han Guo и соавторы
CODA: Rewriting Transformer Blocks as GEMM-Epilogue Programs
arXiv:2605.19269v2 · 20 мая 2026

Узкое место: обмен с памятью вокруг GEMM

GEMM вычисляет $H = XW$ на Tensor Cores.
Нормализация и активации часто ограничены **пропускной способностью памяти** (memory-bound), а не арифметикой.



Fusion объединяет операции в одно GPU-ядро:
результат GEMM обрабатывается **до записи в HBM**.

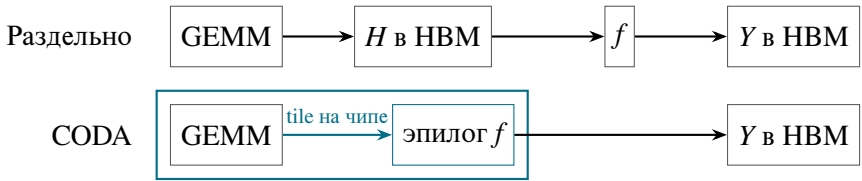
Ускорение низкоточной GEMM делает цену обмена ещё заметнее.

CODA, §1–2; HBM — глобальная память GPU.

Абстракция CODA: GEMM + эпилог

Mainloop умножает матрицы; **эпилог** обрабатывает полученный tile — небольшой блок выхода — на чипе.

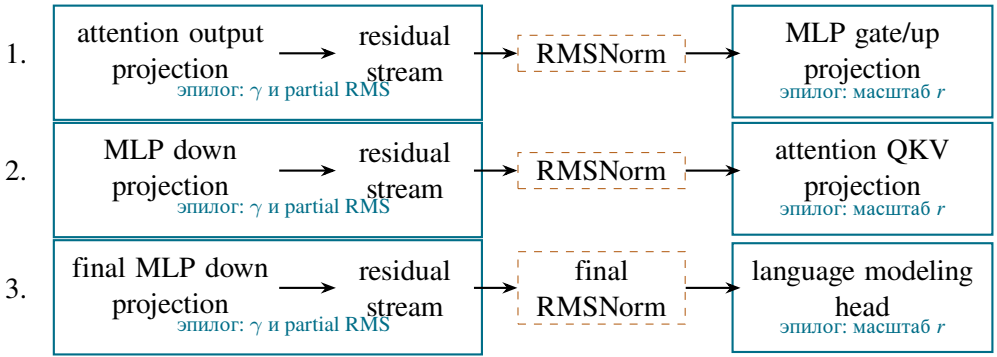
$$H = XW, \qquad Y_{[i,j]} = f_{[i,j]}\left(H_{[i,j]}\right).$$



CODA, §3.1; рамка объединяет mainloop и эпилог в одно ядро.

Где GEMM + эпилог встречается в Transformer

Три канонические цепочки GEMM–residual–RMSNorm–GEMM.

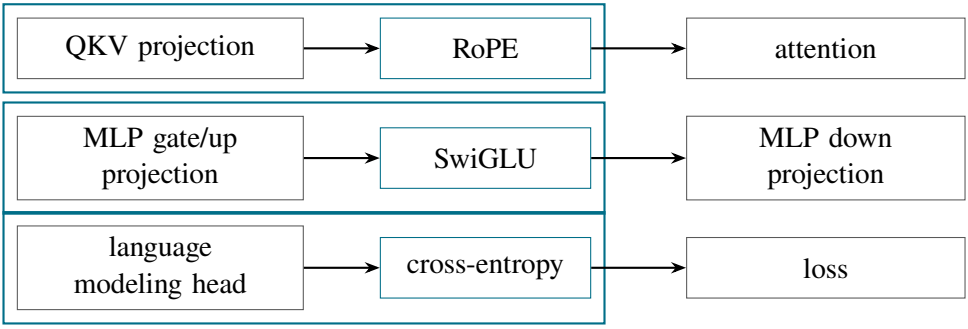


Рамки: GEMM1 + residual + partial RMS; GEMM2 + масштаб r.

RMSNorm распределена между эпилогами; отдельно — только reduce partials.

CODA, §3.2.1; стрелки показывают канонический порядок операций.

Другие цепочки GEMM + эпилог



Рамка: проекция и следующая локальная операция — одно ядро.
RoPE применяется к Q и K; attention выполняется отдельно.

Для cross-entropy в эпилоге вычисляются статистики tile;
итоговая функция потерь получается после малой редукции.

CODA, §3.2.2–3.2.3; отдельная финальная редукция cross-entropy не показана.

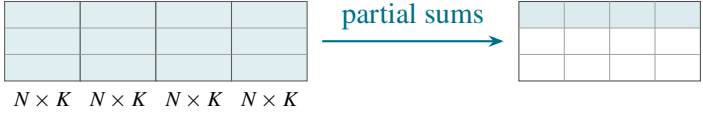
GEMM1: выходные tile и `partial_rms`

$N = N_{\text{tile_size}}$ **— высота tile;** $K = K_{\text{tile_size}}$ **— его ширина.**
d **— ширина активации;** $k = \lceil d/K \rceil$ **— число tile по второй оси.**

$$A = XW_0 + Z \in \mathbb{R}^{N \times d}, \qquad B = A \odot \gamma \in \mathbb{R}^{N \times d}.$$

$P = \texttt{partial_rms}$
$N \times k$

A: блок $N \times d$, разбитый на k tile

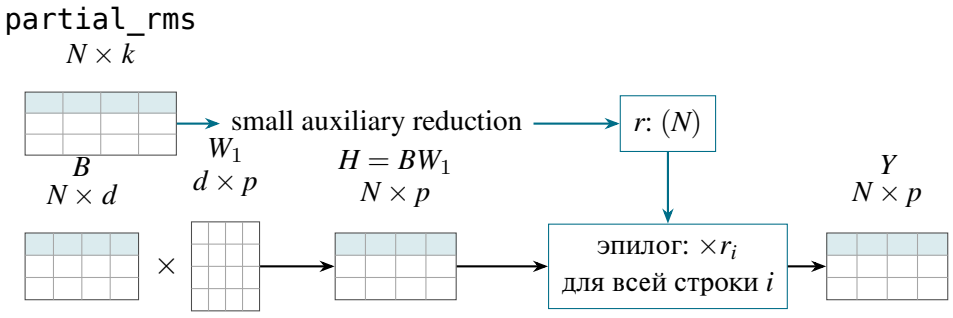


$$P_{it} = \sum_{j \in \text{tile } t} A_{ij}^2, \qquad \boxed{P = \texttt{partial_rms} \in \mathbb{R}^{N \times k}}.$$

Один tile $N \times K$ даёт N чисел; k tile дают матрицу $N \times k$.

CODA, §3.2.1; размеры для одного блока строк; $\gamma \in \mathbb{R}^d$; partial_rms — суммы квадратов.

Редукция до r, затем эпилог второй GEMM



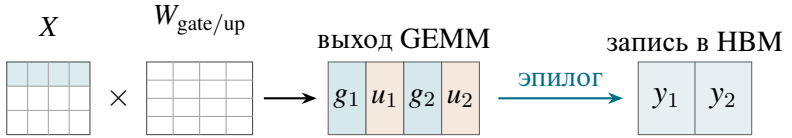
$$r_i = \left(\frac{1}{d} \sum_{t=1}^{k} P_{it} + \varepsilon \right)^{-1/2}, \quad H = BW_1, \quad Y_{i,:} = r_i H_{i,:}.$$

Масштаб один на строку: $(r \odot B)W_1 = r \odot (BW_1)$.

Нормализованный тензор перед GEMM2 отдельно не материализуется.

CODA, §3.2.1; $(N, k) \to (N)$ — редукция по второй оси; N здесь — высота row tile.

Попарные операции: пара уже рядом в регистрах



SwiGLU: $X : N \times d$, $W_{\text{gate/up}} : d \times 2h$;
выход GEMM: $N \times 2h$; gate и value: $N \times h$; $Y : N \times h$.

$$y_j = \text{SiLU}(g_j)u_j, \qquad \text{SiLU}(g) = g\sigma(g).$$

RoPE: $X : N \times d$, $W_{\text{QKV}} : d \times D_{\text{QKV}}$;
выход до и после RoPE: $N \times D_{\text{QKV}}$; поворот пары: 2×2.

$$\begin{pmatrix} a' \\ b' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}.$$

Обе операции локальны: отдельное чтение выхода GEMM не нужно.

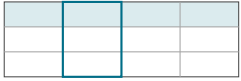
CODA, §3.2.2; h — ширина MLP; D_{QKV} — суммарная ширина Q, K, V.

Cross-entropy: сжать tile до статистик

$X \in \mathbb{R}^{N\times d}$, $W_{\text{lm}} \in \mathbb{R}^{d\times V}$, $H \in \mathbb{R}^{N\times V}$; $\ell \in \mathbb{R}^{N}$.

$$H = XW_{\text{lm}}, \qquad \ell_i = -H_{i,t_i} + \underbrace{\log \sum_k e^{H_{ik}}}_{\text{LSE}_i}.$$

логиты H: токены $\times$ словарь



эпилог tile $\longrightarrow$ (m_{it}, s_{it}), целевой логит $\longrightarrow$ малая редукция LSE_i и ℓ_i

$$m_{it} = \max_{k\in t} H_{ik}, \qquad s_{it} = \sum_{k\in t} e^{H_{ik}-m_{it}}.$$

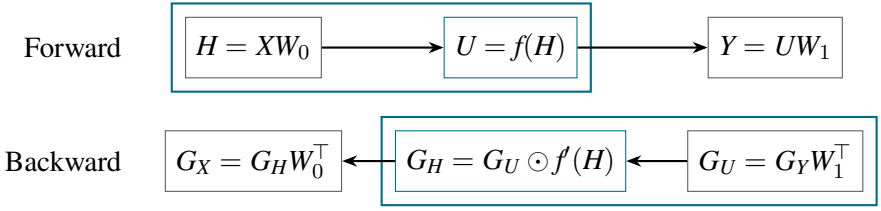
$m, s \in \mathbb{R}^{N\times T}$; T — число tile по словарю V.

Статистики объединяются без отдельного softmax; логиты сохраняются.

CODA, §3.2.3; V — размер словаря; t_i — целевой токен; LSE — log-sum-exp.

Backward: производная становится эпилогом

$X : N \times d$, $W_0 : d \times h$, $H, U : N \times h$, $W_1 : h \times p$, $Y : N \times p$.



В forward эпилог обрабатывает **активацию**.
В backward — **пришедший градиент**; направление fusion меняется.

$$G_{W_1} = U^\top G_Y, \qquad G_{W_0} = X^\top G_H.$$

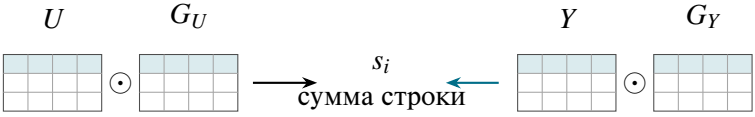
G_T имеет ту же размерность, что и T: $G_{W_0} : d \times h$, $G_{W_1} : h \times p$.
Локальная функция даёт локальную производную; для пары — VJP.

CODA, §3.2.4, теорема 1; $G_T = \partial L / \partial T$, рамка — одно ядро.

RMSNorm backward: та же статистика в другом месте

$U = \mathrm{RMSNorm}(A, \gamma) : N \times d$; $W_1 : d \times p$; $Y = UW_1 : N \times p$.

$$\boxed{s_i = \frac{1}{d}\langle U_{i,:}, G_{U,i,:}\rangle = \frac{1}{d}\langle Y_{i,:}, G_{Y,i,:}\rangle}, \qquad G_U = G_Y W_1^\top.$$



внутри RMSNorm backward — на соседней границе GEMM

Интуиция: скалярный вклад строки не меняется,
если перенести проекцию через скалярное произведение.

$G_U : N \times d$, $G_Y : N \times p$, $s : N$, $G_\gamma : d$.

Статистики и G_γ завершаются малыми редукциями по partial sums.

CODA, §3.2.4, приложение A.2; d — ширина входа RMSNorm.

Результаты и границы ускорения

Один NVIDIA H100; baseline — cuBLAS + `torch.compile`.
Ускорение $S = T_{\text{baseline}}/T_{\text{CODA}}$.

Layer forward + backward	CODA (LLM)	CODA (human)
$d = 2048$	$\approx 1.055\times$	$\approx 1.08\times$
$d = 4096$	$\approx 1.04\times$	$\approx 1.05\times$
$d = 8192$	$\approx 1.005\times$	$\approx 1.035\times$

В отдельных backward-ядрах: примерно 1.6–$1.7\times$.

В последовательности блоков выигрыш скромнее.

«Layer» — два GEMM–RMSNorm паттерна: редукции учтены,
attention и optimizer исключены. Это не полный шаг обучения.
LLM-ядра созданы с сопровождением человека на CuTeDSL.
Перенос масштаба меняет округления в BF16. Числа — оценки по рис. 11.

CODA, §3.3–4, рис. 6, 10–11, приложение C.2.